CGV WEB

Calorimeter Geometry Viewer — Documentação Técnica Completa

Versão 1.0 · Março 2026

Projeto: CGV WEB — Visualizador 3D de Geometria de Calorímetros do ATLAS (CERN)

Instituição: UFJF / NIPSCERN · CERN ATLAS TileCal

Repositório: github.com/nipscernlab/cgv-web

Tecnologias: JavaScript (Node.js + Browser), Rust/WebAssembly, Three.js, JSROOT

Sumário

1. Visão Geral do Projeto

1.1 O que é o CGV WEB

1.2 Estrutura de Arquivos

1.3 Dependências e Tecnologias

2. Formatos de Dados

2.1 Arquivo .root

2.2 Arquivo .cgv

2.3 Arquivo .glb

2.4 XML de Eventos (JiveXML)

3. Pipeline de Conversão: .root → .cgv + .glb

3.1 Visão Geral do Pipeline

3.2 Leitura do Arquivo .root

3.3 Geração do .cgv (Árvore Hierárquica)

3.4 Geração do .glb (Modelo 3D)
3.5 Otimização e Filtragem do GLB

4. Processamento de Eventos XML (WASM/Rust)

4.1 Arquitetura do Módulo WASM
4.2 Parsing do XML
4.3 Mapeamento TileCal
4.4 Mapeamento LAr EM Barrel
4.5 Mapeamento LAr EM Endcap
4.6 Mapeamento HEC
4.7 Mapeamento MBTS
4.8 Tratamento de Hits Duplicados
4.9 Formato da Resposta JSON

5. Viewer Web (index.html)

5.1 Inicialização da Aplicação
5.2 Carregamento da Geometria
5.3 Renderização de Eventos (loadXmlContent)
5.4 Sistema de Colormaps
5.5 Interação e Seleção de Células
5.6 Ghost Atlas e Elementos Auxiliares
5.7 Jets (Cones 3D)

6. Sistema de Eventos ao Vivo

6.1 LivePoller
6.2 CORS Proxy
6.3 Cache de Eventos
6.4 Modo Offline

7. Delivery dos Arquivos

7.1 Estratégia de Cache
7.2 Hosting e CDN
7.3 Validação de Integridade

8. Restrições, Validações e Tolerâncias

9. Setup e Utilitários

10. Internacionalização

1. Visão Geral do Projeto

1.1 O que é o CGV WEB

O **CGV WEB** (Calorimeter Geometry Viewer) é um sistema web de visualização 3D interativo para dados de colisão do detector ATLAS no CERN. O projeto possui dois grandes pilares:

1. **Pipeline de Conversão Offline:** Converte arquivos de geometria CERN `.root` em dois formatos web-friendly: um arquivo de hierarquia textual `.cgv` e um modelo 3D binário `.glb` (GLTF 2.0).
2. **Viewer Web Interativo:** Uma aplicação single-page que carrega a geometria 3D, recebe dados de eventos de colisão via XML (JiveXML), mapeia os hits de energia às células físicas do detector, e renderiza tudo em tempo real com Three.js.

O sistema suporta dados ao vivo do ATLAS (via polling do servidor CERN) e também carregamento local de arquivos XML para análise offline. Cobre os seguintes sub-detectores calorimétricos:

SUB-DETECTOR	ABREVIAÇÃO	COBERTURA H	DESCRIÇÃO
Tile Calorimeter	TileCal / TILE	$\|\eta\| < 1.7$	Calorímetro hadrônico de barril com telhas cintilantes
Minimum Bias Trigger Scintillators	MBTS	$2.1 < \|\eta\| < 3.8$	Cintiladores para trigger de minimum bias
Liquid Argon EM Calorimeter	LAr	$\|\eta\| < 3.2$	Calorímetro eletromagnético de argônio líquido (barrel + endcap)
Hadronic Endcap Calorimeter	HEC	$1.5 < \|\eta\| < 3.2$	Calorímetro hadrônico de endcap

1.2 Estrutura de Arquivos

```
cgv-web/
├── index.html                          # Viewer web principal
(~2930 linhas)
├── root2scene.mjs                      # Conversor .root → .cgv
+ .glb (616 linhas)
├── setup.mjs                           # Pós-instalação: patch
de imports JSROOT
├── filter_glb.mjs                      # Filtra geometria
TileCal do GLB completo
├── optimize_glb.mjs                    # Remove dados não
utilizados e comprime GLB
├── package.json                        # Configuração npm e
dependências
├── lib/
│   ├── polyfill.mjs                    # Polyfill FileReader
para Node.js
│   ├── geobase.mjs                     # Construtor de
geometrias JSROOT (patcheado)
│   └── csg.mjs                         # CSG (Constructive Solid
Geometry, patcheado)
├── wasm/
│   ├── src/lib.rs                      # Código-fonte Rust do
módulo WASM (667 linhas)
│   ├── Cargo.toml                      # Dependências Rust
│   └── pkg/                            # Saída compilada do WASM
│       ├── tile_viz.js                 # Wrapper JavaScript para
o WASM
│       ├── tile_viz_bg.wasm            # Módulo WebAssembly
binário
│       └── tile_viz.d.ts               # Definições TypeScript
├── xml_test/live_cern/
│   ├── live_poller.js                  # Poller de eventos ao
vivo do ATLAS (280 linhas)
│   └── cors-proxy.js                   # CORS proxy para
Cloudflare Workers
└── root_file/
    ├── CaloGeometry.glb                # Geometria 3D pré-gerada
    └── CaloGeometry.cgv                # Hierarquia textual pré-
gerada
```

1.3 Dependências e Tecnologias

COMPONENTE	TECNOLOGIA	USO
Leitura .root	JSROOT (^7.10.3)	Parser de arquivos ROOT do CERN
Geometria 3D	Three.js (0.162.0 via CDN)	Renderização WebGL, meshes, materiais
Exportação GLB	GLTFExporter (Three.js addon)	Serialização da cena para GLTF 2.0 binário
Processamento de hits	Rust → WebAssembly	Matching de alta performance de hits a células
Otimização GLB	@gltf-transform/core + functions	Deduplicação, poda, quantização
CORS Proxy	Cloudflare Workers	Bypass CORS para dados ATLAS ao vivo

2. Formatos de Dados

2.1 Arquivo .root

O arquivo `.root` é o formato padrão de armazenamento de dados do framework ROOT (CERN). No contexto do CGV WEB, contém a **geometria completa do detector calorimétrico** na forma de um objeto `TGeoManager` (ou `TGeoVolume`), que encapsula uma árvore hierárquica de volumes geométricos.

Cada nó da árvore representa um volume físico do detector e contém:

- `fName` — Identificador do volume (ex: `"Tile1p"`, `"EMBarrel0p"`)
- `fVolume` — Referência ao `TGeoVolume` com shape e filhos
- `fShape` — Primitiva geométrica (TGeoBBox, TGeoTube, TGeoTubeSeg, etc.)
- `fMatrix` — Transformação local (translação, rotação, escala)
- `fNodes` — Array de nós filhos (`TGeoNode`)
- `fGeoAtt` — Atributos de visibilidade (bit 0x08 = kVisThis)
- `fFillColor`, `fLineColor` — Cores ROOT (índices de paleta)

2.2 Arquivo .cgv

O formato `.cgv` (Calorimeter Geometry Viewer) é um **arquivo de texto flat** que codifica a hierarquia completa de volumes do detector. Cada linha representa um caminho na árvore, do nó raiz até uma folha.

Estrutura do formato:

```
# Calorimeter Geometry Viewer (CGVWEB)
# source    : CaloGeometry.root
# generated: 2026-03-24T12:34:56.789Z
# format    : name \t → \t name \t → \t … \t → \t leaf

Calorimeter
Calorimeter    →      Tile1p
Calorimeter    →      Tile1p →      Tile1p0_0
Calorimeter    →      Tile1p →      Tile1p0_0     →      cell_0
Calorimeter    →      Tile1p →      Tile1p0_0     →      cell_1
...
Calorimeter    →      EMBarrel0p    →      EMBarrel0p0_0 →
cell_0
```

Regras do formato:

- Linhas começando com `#` são comentários (ignoradas pelo parser)
- Separador de caminho: `TAB → TAB` (caractere U+2192, seta para direita)
- Apenas linhas terminando com `cell_*` são contadas como células reais
- A correspondência de nomes é *case-sensitive*
- O caminho completo serve como **chave única** para mapear hits a meshes 3D

> **Importância:** O caminho CGV é a "cola" entre o WASM (que gera paths como `Calorimeter→Tile1p_0→Tile1p0_0→cell_5`) e o viewer (que indexa meshes pelo nome no GLB).

2.3 Arquivo .glb

O `.glb` é o formato binário do padrão GLTF 2.0 (GL Transmission Format). No CGV WEB, cada mesh no GLB corresponde a uma **célula individual do detector**.

Características do GLB gerado:

- Cada `THREE.Mesh` tem `mesh.name = caminho CGV completo`
- Geometrias idênticas são **compartilhadas** (deduplicação por shape signature)
- Transformações são decompostas em posição (T), rotação (R) e escala (S)

- Materiais incluem cor ROOT e opacidade extraída de `fFillStyle`
- O formato binário é compacto e carregável diretamente pelo Three.js via `GLTFLoader`

2.4 XML de Eventos (JiveXML)

Os eventos de colisão são fornecidos no formato **JiveXML**, padrão usado pelo ATLAS Event Display. Cada arquivo contém hits de energia depositados nos calorímetros.

Estrutura esperada do XML:

```
<Event>
  <TILE count="5200">
    <energy>100.5 200.3 150.2 ...</energy>
    <eta>0.05 -0.15 0.25 ...</eta>
    <phi>1.57 -2.1 0.8 ...</phi>
    <sub>3 2 5 ...</sub>
  </TILE>

  <MBTS count="32">
    <energy>50 75 ...</energy>
    <channel>0 1 ...</channel>
    <module>2 3 ...</module>
    <eta>2.4 -2.4 ...</eta>
    <phi>0 0.78 ...</phi>
  </MBTS>

  <LAr count="12000">
    <energy>120 140 ...</energy>
    <eta>0.5 0.6 ...</eta>
    <phi>1.2 2.3 ...</phi>
    <slot>0 1 ...</slot>
  </LAr>

  <HEC count="3000">
    <energy>180 200 ...</energy>
    <eta>1.8 1.9 ...</eta>
    <phi>1.5 2.0 ...</phi>
    <slot>0 1 ...</slot>
  </HEC>

  <Jet storeGateKey="AntiKt4EMTopoJets_xAOD" count="5">
    <eta>0.5 -1.2 ...</eta>
    <phi>1.5 -0.8 ...</phi>
    <et>150 80 ...</et>
    <energy>200 120 ...</energy>
  </Jet>
</Event>
```

Cada sub-detector possui arrays de valores separados por espaço. Os arrays devem ter o **mesmo comprimento** — em caso de discrepância, o sistema trunca ao menor comprimento.

3. Pipeline de Conversão: .root → .cgv + .glb

3.1 Visão Geral do Pipeline

O script `root2scene.mjs` é o ponto de entrada para a conversão. Ele é executado via Node.js e produz dois arquivos de saída a partir de um único arquivo `.root`:

```
          ┌──────────────┐
          │  .root file  │
          └──────┬───────┘
                 │
          ┌──────▼───────┐
          │   JSROOT     │  openFile() → readObject()
          │   openFile   │  Encontra TGeoManager/TGeoVolume
          └──────┬───────┘
                 │
        ┌────────┴────────┐
        │                 │
   ┌────▼────┐       ┌────▼────┐
   │ buildCgv│       │buildGltf│
   │         │       │         │
   │ Árvore  │       │ Three.js│
   │ textual │       │ Meshes  │
   └────┬────┘       └────┬────┘
        │                 │
   ┌────▼────┐       ┌────▼──────────┐
   │ .cgv    │       │ GLTFExporter  │
   │ (texto) │       │     ↓         │
   └─────────┘       │  .glb         │
                     │ (binário)     │
                     └───────────────┘
```

Uso via linha de comando:

```
node root2scene.mjs CaloGeometry.root --out ./root_file --
visible-only --verbose
```

OPÇÃO	DESCRIÇÃO
`--out <dir>`	Diretório de saída (padrão: mesmo do .root)
`--max-faces <n>`	Limite de faces por shape (0 = ilimitado)
`--depth <n>`	Profundidade máxima da árvore (0 = completa)
`--subtree <prefix>`	Filtra filhos diretos da raiz por prefixo de nome
`--visible-only`	Ignora volumes invisíveis (bit kVisThis = 0x08)
`--no-gltf`	Gera apenas o .cgv
`--no-cgv`	Gera apenas o .glb
`--verbose`	Log detalhado de cada nó processado

3.2 Leitura do Arquivo .root

A leitura é feita em duas etapas no arquivo `root2scene.mjs`:

Etapa 1: Abertura do arquivo

A função `openRootFile()` (linha 121) utiliza a biblioteca JSROOT para abrir o arquivo binário e listar suas chaves:

```
// root2scene.mjs — openRootFile (linha 121)
async function openRootFile(path) {
  const file = await openFile(path);    // JSROOT abre o binário
ROOT
  if (!file) die(`Não foi possível abrir: ${path}`);
  const keys = Array.isArray(file.fKeys) ? file.fKeys : [];
  info(`${keys.length} chave(s) no arquivo`);
  return { file, keys };
}
```

Etapa 2: Localização do TGeoManager

A função `findGeoManager()` (linha 130) percorre as chaves do arquivo procurando um objeto de geometria:

```
// root2scene.mjs — findGeoManager (linha 130)
async function findGeoManager(file, keys) {
  const GEO = new Set(['TGeoManager', 'TGeoVolume',
'TGeoVolumeAssembly']);
  for (const key of keys) {
    if (GEO.has(key.fClassName)) {
      const obj = await file.readObject(key.fName, key.fCycle);
      return { obj, key };
    }
  }
  return null;
}
```

O sistema aceita três tipos de objetos raiz: `TGeoManager`, `TGeoVolume` e `TGeoVolumeAssembly`. Se for um `TGeoManager`, o volume raiz é acessado via `obj.fMasterVolume`.

3.3 Geração do .cgv (Árvore Hierárquica)

A função `buildCgv()` (linha 374 de `root2scene.mjs`) realiza uma **travessia em profundidade baseada em pilha** da árvore de nós geométricos, emitindo uma linha para cada nó visitado.

```
// root2scene.mjs — buildCgv (simplificado, linha 374)
function buildCgv(geoResult, allKeys, rootPath, opts) {
  const lines = [];
  lines.push(`# Calorimeter Geometry Viewer (CGVWEB)`);
  lines.push(`# source   : ${basename(rootPath)}`);

  const topNode = (obj._typename === 'TGeoManager')
    ? { fName: key.fName, fVolume: obj.fMasterVolume, fGeoAtt:
0xFF, fMatrix: null }
    : obj;

  const stack = [{ node: topNode, ancestors: [] }];

  while (stack.length > 0) {
    const { node, ancestors } = stack.pop();
    const volume = node.fVolume ?? node;
    const geoAtt = node.fGeoAtt ?? volume?.fGeoAtt ?? 0xFF;

    // Filtragem de visibilidade
    if (opts.visibleOnly && !Boolean(geoAtt & 0x08)) continue;

    const name = node.fName ?? volume?.fName ?? '?';
    const segments = [...ancestors, name];
    lines.push(segments.join('\t→\t'));     // Emite: "A → B → C"

    const children = volume?.fNodes?.arr ?? [];
    if (children.length > 0 && (opts.maxDepth === 0 ||
ancestors.length < opts.maxDepth)) {
      for (let i = children.length - 1; i >= 0; i--) {
        stack.push({ node: children[i], ancestors: segments });
      }
    }
  }

  return lines.join('\n') + '\n';
}
```

Passo a passo do algoritmo:

1. Cria o nó raiz (topNode) a partir do TGeoManager ou TGeoVolume
2. Inicializa uma pilha com o nó raiz e ancestors vazio
3. Para cada nó retirado da pilha:
   - Verifica visibilidade via bit `geoAtt & 0x08`
   - Constrói o caminho completo juntando ancestors + nome atual com

`\t→\t`

- Emite a linha no arquivo de saída
- Empilha os filhos em ordem reversa (para manter ordem natural na travessia)
- Respeita o limite de profundidade (`--depth`)

3.4 Geração do .glb (Modelo 3D)

A função `buildGltf()` (linha 460 de `root2scene.mjs`) constrói uma cena Three.js onde cada célula do detector é um mesh nomeado. O processo envolve múltiplos subsistemas.

3.4.1 Deduplicação de Geometrias (Shape Signature)

Para evitar duplicação de BufferGeometry para shapes idênticas, a função `shapeSignature()` (linha 248) gera uma chave única baseada nos parâmetros da shape:

```
// root2scene.mjs — shapeSignature (linha 248)
function shapeSignature(shape) {
  const t = shape._typename ?? '';
  switch (t) {
    case 'TGeoBBox':
      return `BBox_${r4(shape.fDX)}_${r4(shape.fDY)}
_${r4(shape.fDZ)}`;
    case 'TGeoTube':
      return `Tube_${r4(shape.fRmin)}_${r4(shape.fRmax)}
_${r4(shape.fDZ)}`;
    case 'TGeoTubeSeg':
      return `TSeg_${r4(shape.fRmin)}_${r4(shape.fRmax)}
_${r4(shape.fDZ)}_${r4(shape.fPhi1)}_${r4(shape.fPhi2)}`;
    // ... outras primitivas
    default:
      return `${t}_rnd_${Math.random()}`;  // Shapes complexas:
sem compartilhamento
  }
}
```

Shapes que produzem a mesma signature compartilham a mesma instância de `BufferGeometry` no GLB, resultando em redução significativa de tamanho.

3.4.2 Extração de Cores ROOT

A função `volumeColorThree()` (linha 184) converte cores do ROOT para Three.js:

```
// root2scene.mjs — volumeColorThree (linha 184)
function volumeColorThree(volume, rootColors) {
  const { hex, opacity } = volumeColorCss(volume, rootColors);
  return { color: new THREE.Color(hex), opacity };
}

function volumeColorCss(volume, rootColors) {
  let hex = '#888888', opacity = 1.0;
  // 1. Tenta fFillColor
  if ((volume.fFillColor ?? 0) > 1)
    hex = colorHex(volume.fFillColor, rootColors);
  // 2. Fallback: fLineColor
  else if ((volume.fLineColor ?? -1) >= 0)
    hex = colorHex(volume.fLineColor, rootColors);
  // 3. Opacidade via fFillStyle (faixa 3000-3100)
  const mat = volume.fMedium?.fMaterial;
  if (mat) {
    const fs = mat.fFillStyle ?? 0;
    if (fs >= 3000 && fs <= 3100)
      opacity = (100 - (fs - 3000)) / 100;
  }
  return { hex, opacity };
}
```

3.4.3 Transformações de Matrizes

A função `nodeToMatrix4()` (linha 189) converte as matrizes de transformação ROOT para `THREE.Matrix4`:

```
// root2scene.mjs — nodeToMatrix4 (linha 189)
function nodeToMatrix4(node) {
  const m = node?.fMatrix;
  if (!m) return null;

  switch (m._typename) {
    case 'TGeoTranslation':
      translation = m.fTranslation; break;
    case 'TGeoRotation':
      rotation = m.fRotationMatrix; break;
    case 'TGeoCombiTrans':
      translation = m.fTranslation;
      rotation = m.fRotation?.fRotationMatrix; break;
    case 'TGeoHMatrix':
      translation = m.fTranslation;
      rotation = m.fRotationMatrix;
      scale = m.fScale; break;
    case 'TGeoIdentity':
    default:
      return null;    // Identidade → sem transformação
  }

  const mat4 = new THREE.Matrix4();
  if (rotation?.length === 9)
    mat4.set(rotation[0], rotation[1], rotation[2], 0, ...);
  if (translation?.length >= 3)
    mat4.setPosition(translation[0], translation[1],
translation[2]);
  if (scale?.length >= 3)
    mat4.scale(new THREE.Vector3(scale[0], scale[1], scale[2]));

  return mat4;
}
```

3.4.4 Construção dos Meshes

O loop principal de `buildGltf()` percorre a árvore acumulando transformações e criando meshes:

```
// root2scene.mjs — buildGltf loop principal (simplificado, linha
487)
const stack = [{ node: topNode, worldMat: identity, ancestorPath:
'', depth: 0 }];

while (stack.length > 0) {
  const { node, worldMat, ancestorPath, depth } = stack.pop();
  const shape = volume?.fShape;
  const path = ancestorPath ? `${ancestorPath}→${name}` : name;

  // Compõe transformação acumulada
  const local   = nodeToMatrix4(node);
  const nodeMat = local ? worldMat.clone().multiply(local) :
worldMat;

  if (shape) {
    // Reutiliza geometria se signature já existe
    let bufGeo = geoCache.get(shapeSignature(shape));
    if (!bufGeo)
      bufGeo = createGeometry(shape, opts.maxFaces);  //
geobase.mjs

    if (bufGeo) {
      const { color, opacity } = volumeColorThree(volume,
rootColors);
      const mat  = matCache.get(color, opacity);
      const mesh = new THREE.Mesh(bufGeo, mat);

      mesh.name = path;  // Nome = caminho CGV completo

      // Decomposição TRS para posicionamento
      const pos = new THREE.Vector3(), quat = new
THREE.Quaternion();
      nodeMat.decompose(pos, quat, scl);
      mesh.position.copy(pos);
      mesh.quaternion.copy(quat);

      scene.add(mesh);
    }
  }

  // Empilha filhos com transformação acumulada
  for (const child of children.reverse())
    stack.push({ node: child, worldMat: nodeMat, ancestorPath:
path, depth: depth + 1 });
}
```

Ao final, a cena é exportada como GLB binário via `GLTFExporter`:

```
const exporter = new GLTFExporter();
const glbArrayBuffer = await exporter.parseAsync(scene, {
  binary: true, embedImages: false, onlyVisible: false
});
return Buffer.from(glbArrayBuffer);
```

3.4.5 Cache de Materiais

A classe `MaterialCache` (linha 442) garante que materiais com mesma cor e opacidade são reutilizados:

```
// root2scene.mjs — MaterialCache (linha 442)
class MaterialCache {
  #map = new Map();
  get(color, opacity) {
    const key = `${color.getHexString()}_${opacity.toFixed(3)}`;
    if (!this.#map.has(key)) {
      this.#map.set(key, new THREE.MeshStandardMaterial({
        color, opacity,
        transparent: opacity < 1.0,
        roughness: 0.55, metalness: 0.05,
        side: THREE.DoubleSide,
      }));
    }
    return this.#map.get(key);
  }
}
```

3.5 Otimização e Filtragem do GLB

filter_glb.mjs

O script `filter_glb.mjs` remove nós não-TileCal do GLB completo. Utiliza `@gltf-transform` para percorrer os nós e remover aqueles cujo nome contém `HEC`, `EMBarrel`, `EMEndCap`, etc.:

```
// filter_glb.mjs (simplificado)
function shouldRemove(node) {
  const n = node.getName();
  if (!n.includes('Tile') && (
    n.includes('HEC') || n.includes('EMBarrel') ||
n.includes('EMEndCap') ||
    n.includes('Ground') || n === 'Calorimeter→I'
  )) return true;
  return false;
}

// Após remoção, poda recursos não referenciados
await doc.transform(prune(), dedup());
```

optimize_glb.mjs

O script `optimize_glb.mjs` reduz o tamanho do GLB removendo dados desnecessários para o viewer:

1. **Strip de atributos de vértice:** TEXCOORD_0, TEXCOORD_1, TANGENT, COLOR_0 (o viewer usa cor por instância)
2. **Remoção de materiais:** O viewer substitui todos por `baseMat`
3. **Deduplicação:** Accessors e buffers idênticos são mesclados
4. **Quantização (opcional):** Float32 → Int16 com fator de escala (flag `--quantize`)

Resultado típico: **~80% de redução** no tamanho do arquivo.

4. Processamento de Eventos XML (WASM/Rust)

4.1 Arquitetura do Módulo WASM

O módulo WebAssembly é compilado a partir de Rust (`wasm/src/lib.rs`) e expõe duas funções públicas ao JavaScript:

FUNÇÃO	PARÂMETROS	RETORNO	DESCRIÇÃO
`load_cgv(cgv_text)`	String com conteúdo CGV	usize (nº de células)	Conta células no CGV para validação
`process_event(xml_text)`	String com XML do evento	String JSON	Processa hits e retorna mapeamento

O módulo é carregado no viewer via:

```
// index.html (linha 1047)
import init, { load_cgv, process_event } from './wasm/pkg/
tile_viz.js';
// Na inicialização:
await init();  // Carrega e inicializa o .wasm
```

4.2 Parsing do XML

O parser XML é implementado diretamente em Rust com funções de busca de texto, sem dependência de um parser XML completo:

```
// wasm/src/lib.rs — extract_array (linha 630)
fn extract_array(content: &str, tag: &str) -> Option<Vec<f32>> {
    let open  = format!("<{}>",  tag);
    let close = format!("</{}>", tag);
    let s = content.find(&open)?  + open.len();
    let e = content[s..].find(&close)? + s;
    Some(content[s..e].split_whitespace()
        .filter_map(|t| t.parse().ok())    // Ignora tokens não-
numéricos
        .collect())
}

// parse_calo_arrays: extrai bloco XML e retorna 4 arrays
fn parse_calo_arrays(xml_text: &str, tag: &str, fields: &[&str])
    -> Option<(Vec<f32>, Vec<f32>, Vec<f32>, Vec<f32>)>
{
    let pos = find_tag(xml_text, tag)?;           // Encontra
<TAG
    let inner_start = xml_text[pos..].find('>')? + pos + 1;
    let close_tag = format!("</{}>", tag);
    let inner_end = xml_text[inner_start..].find(&close_tag)? +
inner_start;
    let inner = &xml_text[inner_start..inner_end];

    let a = extract_array(inner, fields[0]).unwrap_or_default();
    let b = extract_array(inner, fields[1]).unwrap_or_default();
    let c = extract_array(inner, fields[2]).unwrap_or_default();
    let d = extract_array(inner, fields[3]).unwrap_or_default();
    Some((a, b, c, d))
}
```

Fluxo de parsing para cada sub-detector:

1. Localiza a tag de abertura (ex: `<TILE`) no XML
2. Extrai o conteúdo entre `<TILE>` e `</TILE>`
3. Para cada campo (energy, eta, phi, sub), extrai o texto entre as sub-tags
4. Divide por whitespace e parseia para `f32`
5. Tokens inválidos são silenciosamente ignorados (`filter_map`)
6. O tamanho efetivo `n` é o mínimo de todos os arrays

4.3 Mapeamento TileCal

O TileCal é mapeado usando uma **tabela estática de 64 células** definida no

início do arquivo Rust. Cada entrada define:

```
// wasm/src/lib.rs (linha 9)
struct TileCell {
    sub:       u8,           // Sub-detector (0-7)
    eta_c:     f32,          // η central da célula
    tile_vol:  &'static str, // Nome do volume no CGV (ex:
"Tile1p")
    eta_i:     u8,           // Índice η
    phi_n:     u8,           // Número de setores ϕ (64 ou 8)
    cell_name: &'static str, // Rótulo (ex: "A1", "BC5", "D3")
}
```

Sub-detectores TileCal:

SUB	NOME	H	VOLUMES CGV
3	LBA (Long Barrel A-side)	η > 0	Tile1p, Tile23p, Tile4p
2	LBC (Long Barrel C-side)	η < 0	Tile1n, Tile23n, Tile4n
5	EBA regular (Extended Barrel A)	η > 0	Tile5p–Tile8p
0	EBC regular (Extended Barrel C)	η < 0	Tile5n–Tile8n
4	EBA especial	η > 0	Tile9p–Tile13p
1	EBC especial	η < 0	Tile9n–Tile13n
6	MBTS-A	η > 0	Tile14p, Tile15p
7	MBTS-C	η < 0	Tile14n, Tile15n

Algoritmo de lookup (lookup_tile_cell_nth):

```
// wasm/src/lib.rs — lookup_tile_cell_nth (linha 114)
fn lookup_tile_cell_nth(hit_eta: f32, hit_phi: f32, hit_sub: u8,
nth: usize)
    -> Option<(String, &'static str)>
{
    // 1. Filtra células do sub-detector correspondente
    // 2. Encontra a distância mínima em η
    let min_dist = TILE_CELLS.iter()
        .filter(|c| c.sub == hit_sub)
        .map(|c| (c.eta_c - hit_eta).abs())
        .fold(f32::INFINITY, f32::min);

    // 3. Seleciona candidatos dentro da tolerância
    let tol = min_dist + 0.0005;
    let candidates: Vec<&TileCell> = TILE_CELLS.iter()
        .filter(|c| c.sub == hit_sub && (c.eta_c - hit_eta).abs()
<= tol)
        .collect();

    // 4. Seleciona pelo índice nth (para hits duplicados)
    let cell = candidates[nth % candidates.len()];

    // 5. Calcula setor ϕ (0..phi_n-1)
    let raw = ((hit_phi + PI) * phi_n / (2.0 * PI)).floor() as
i32;
    let j = raw.rem_euclid(cell.phi_n as i32) as usize;

    // 6. Gera o caminho CGV completo
    Some((
        format!("Calorimeter→{vol}_0→{vol}{i}_{i}→cell_{j}"),
        cell.cell_name
    ))
}
```

Passo a passo:

1. Filtra a tabela TILE_CELLS pelo sub-detector do hit
2. Calcula a distância |η_cell - η_hit| para cada célula do sub
3. Encontra a distância mínima e aplica tolerância de +0.0005
4. Seleciona candidatos dentro da tolerância
5. Usa o parâmetro `nth` para resolver hits duplicados (mesmo η/φ)
6. Calcula o setor φ: `j = floor((ϕ + π) × ϕ_n / 2π) mod ϕ_n`
7. Retorna o caminho CGV:

`Calorimeter→TileVol_0→TileVolIdx_Idx→cell_j`

4.4 Mapeamento LAr EM Barrel

O LAr Barrel possui 4 camadas com segmentação variável em η:

CAMADA	NOME	Nº CÉLULAS H	ΔH	SETORES Φ
0	PS (Presampler)	61	0.025	64
1	S1 (Strips)	449	~0.003125 (fine) + 1 coarse	64
2	S2 (Middle)	52	0.025	256
3	S3 (Back)	24	0.05	256

Os valores η centrais são calculados algoritmicamente a partir de constantes geométricas (`CaloGeoConst.h` do ATLAS):

```
// wasm/src/lib.rs — laba_eta (linha 150)
fn laba_eta(layer: usize, i: usize) -> f32 {
    match layer {
        0 => 0.0125 + 0.025 * i as f32,        // 61 cells,
Δη=0.025
        1 => {
            if i < 448 { 0.001_562_5 + 0.003_125 * i as f32 }
            else { 1.4125_f32 }                  // última célula
(coarse)
        }
        2 => 0.0125 + 0.025 * i as f32,         // 52 cells,
Δη=0.025
        3 => 0.025  + 0.05  * i as f32,          // 24 cells,
Δη=0.05
        _ => 0.0,
    }
}
```

Algoritmo de lookup LAr Barrel:

1. Verifica se |η| ≤ 1.55 (cobertura do barrel)
2. Para cada uma das 4 camadas:

- Estima o índice da célula via aritmética: `est = round((|η| - base) / delta)`
- Busca em vizinhança ±1 do estimado (O(1) em vez de busca linear)
- Calcula distância e verifica se está dentro da tolerância

3. Filtra candidatos: `dist ≤ best_dist + 0.001`
4. Ordena por distância e depois por camada
5. Seleciona o candidato pelo parâmetro `nth`
6. Calcula setor φ e gera caminho: `Calorimeter→EMBarrel{L}{side}_0→EMBarrel{L}{side}{i}_{i}→cell_{j}`

4.5 Mapeamento LAr EM Endcap

O Endcap cobre 1.35 < |η| < 3.25, com segmentação não-uniforme:

```
// wasm/src/lib.rs — laeb_eta para camada 1 (exemplo, linha 239)
fn laeb_eta(layer: usize, i: usize) -> f32 {
    match layer {
        1 => match i {
            0 => 1.40828,
            1 => 1.44578,
            2 => 1.47078,
            3 => 1.49578,
            4..=99   => 1.509_84 + 0.003_125   * (i - 4) as f32,
// fine
            100..=147 => 1.810_36 + 0.004_166_67 * (i - 100) as
f32, // medium
            148..=211 => 2.011_41 + 0.006_25     * (i - 148) as
f32, // coarse
            _         => 2.420_78 + 0.025        * (i - 212) as
f32, // very coarse
        },
        // ...
    }
}
```

A busca no endcap usa scan linear (tamanhos pequenos: máximo 216 células) e tolerância variável por camada e índice.

4.6 Mapeamento HEC

O HEC (Hadronic Endcap) usa uma tabela de posições radiais `HECZ[7][14]` para calcular η via a relação geométrica:

```
// wasm/src/lib.rs — r_z_to_eta (linha 365)
fn r_z_to_eta(r: f32, z: f32) -> f32 {
    let theta = r.atan2(z);
    -(theta * 0.5).tan().ln()
}
```

O HEC é dividido em 4 volumes lógicos, que combinam até 2 camadas reais:

VOLUME	CÉLULAS H	TIPO	CAMADAS REAIS
HEC1	14	BuildHEC	Camada 0
HEC23	13	MergeHEC	Camadas 1+2
HEC45	12	MergeHEC	Camadas 3+4
HEC67	12	MergeHEC	Camadas 5+6

Para volumes **MergeHEC**, o η é calculado como a média dos η das duas camadas componentes. A tolerância de matching é de **0.08 em η**.

4.7 Mapeamento MBTS

O MBTS é processado com parser dedicado que extrai arrays de `energy`, `channel`, `module`, `eta` e `phi`:

```
// wasm/src/lib.rs — process_event, seção MBTS (linha 494)
let (vol, cell_name, eta_out) = match ch {
    0 => (format!("Tile14{side}"), "MBTS2", 2.40),
    1 => (format!("Tile15{side}"), "MBTS1", 3.20),
    _ => continue,  // channel inválido: ignorado
};
let path = format!("Calorimeter→{vol}_0→{vol}0_0→cell_{md}");
```

O MBTS mapeia diretamente pelo `channel` (0 = MBTS2, 1 = MBTS1) e usa o `module` como índice da célula φ.

4.8 Tratamento de Hits Duplicados

O XML pode conter múltiplos hits com os mesmos (η, φ) — por exemplo, hits em diferentes camadas de amostragem. O sistema usa um `HashMap` de contagem para selecionar candidatos distintos:

```
// wasm/src/lib.rs — Tratamento de duplicatas (exemplo TILE,
linha 468)
let mut seen: HashMap<(i32, u8, u8), usize> = HashMap::new();
for i in 0..n {
    let eta_key  = (hit_eta * 1000.0).round() as i32;
    let phi_j_key = ((hit_phi + PI) * 64.0 / (2.0 * PI)).floor()
as u8;
    let seen_key = (eta_key, phi_j_key, hit_sub);
    let nth = *seen.get(&seen_key).unwrap_or(&0);
    seen.insert(seen_key, nth + 1);

    // nth é passado para a função de lookup, que seleciona o
    // nth-ésimo candidato da lista ordenada
    match lookup_tile_cell_nth(hit_eta, hit_phi, hit_sub, nth) {
... }
}
```

Para LAr e HEC, a chave de duplicação é `(eta_key * 10000, phi_key * 1000)`, proporcionando resolução mais fina.

4.9 Formato da Resposta JSON

A função `process_event()` retorna uma string JSON com a seguinte estrutura:

```
{
  "ok": true,
  "cells_mapped": 5832,
  "cells_unmapped": 48,
  "e_min": -245.3400,
  "e_max": 18420.5000,
  "tile_mapped": 3200,
  "tile_unmapped": 20,
  "mbts_mapped": 32,
  "lar_mapped": 2100,
  "lar_unmapped": 25,
  "hec_mapped": 500,
  "hec_unmapped": 3,
  "hits": [
    {
      "path": "Calorimeter→Tile1p_0→Tile1p0_0→cell_5",
      "energy": 1250.3000,
      "eta": 0.0500,
      "phi": 0.4908,
      "cell": "A1",
      "det": "TILE"
    },
    ...
  ]
}
```

5. Viewer Web (index.html)

5.1 Inicialização da Aplicação

A aplicação é inicializada pela função `initApp()` (linha 2798 de `index.html`) que segue uma sequência precisa de passos:

```
    ┌──────────────────┐
    │ 1. Detectar idioma│ → localStorage / navigator.language
    └──────────┬───────┘
               ▼
    ┌──────────┴───────┐
    │ 2. Carregar WASM  │ → await init()
[20%]
    └──────────┬───────┘
               ▼
    ┌──────────┴──────────────────┐
    │ 3. Download em paralelo:    │
    │    - GLB (via Cache API)    │  fetch com progresso
[55%]
    │    - CGV (texto)            │
    └──────────┬──────────────────┘
               ▼
    ┌──────────┴───────┐
    │ 4. Validar GLB    │ → magic bytes 'glTF'
    └──────────┬───────┘
               ▼
    ┌──────────┴───────┐
    │ 5. Indexar CGV    │ → load_cgv() WASM — contar células
[65%]
    └──────────┬───────┘
               ▼
    ┌──────────┴──────────────────┐
    │ 6. Construir cena 3D:       │
    │    - GLTFLoader.parse()     │
    │    - Indexar meshes por nome│
[90%]
    │    - Configurar câmera      │
    │    - Ghost Atlas            │
    │    - Beam Axis              │
    └──────────┬──────────────────┘
               ▼
    ┌──────────┴───────┐
    │ 7. Iniciar poller │ → Buscar eventos ATLAS
[100%]
    └──────────────────┘
```

```
// index.html — initApp (simplificado, linha 2798)
async function initApp(){
  applyLang(storedLang);

  // 1. WASM
  await init();

  // 2. Download paralelo de geometria
  const [glbBuf, cgvText] = await Promise.all([
    cachedArrayBuffer(GLB_URL, onProgress),
    cachedText(CGV_URL),
  ]);

  // 3. Validação GLB (magic bytes)
  const h = new Uint8Array(glbBuf, 0, 12);
  const ok = h[0]===0x67 && h[1]===0x6C && h[2]===0x54 &&
h[3]===0x46; // 'glTF'
  if (!ok) throw new Error('Not a valid GLB binary');

  // 4. Contar células via WASM
  const cellCount = load_cgv(cgvText);
  totalCellCount = cellCount;

  // 5. Parse GLB e indexar meshes
  const gltf = await new Promise((res, rej) =>
    new GLTFLoader().parse(glbBuf, '', res, rej)
  );
  gltf.scene.traverse(o => {
    if (o.isMesh) {
      o.material = baseMat;
      o.visible = false;
      meshIndex[o.name] = o;  // Chave = caminho CGV
    }
  });

  // 6. Câmera e elementos auxiliares
  const box    = new THREE.Box3().setFromObject(gltf.scene);
  const center = box.getCenter(new THREE.Vector3());
  buildGhostAtlas(1.0, new THREE.Vector3(0, 0, 0));
  buildBeamAxis(new THREE.Vector3(0, 0, 0), 1.0, size * 0.5);

  // 7. Boot poller
  await poller.init();
  poller.start();
}
```

5.2 Carregamento da Geometria

Estratégia de URL:

```
// index.html (linha 1054)
const _isLocal = location.hostname === 'localhost' ||
location.protocol === 'file:';
const GLB_URL = _isLocal
  ? './root_file/CaloGeometry.glb'
  : 'https://media.githubusercontent.com/media/nipscernlab/cgv-
web/main/root_file/CaloGeometry.glb';
const CGV_URL = './root_file/CaloGeometry.cgv';
```

Em produção (GitHub Pages), o GLB é servido via `media.githubusercontent.com` (Git LFS com CORS), enquanto o CGV (texto leve) é servido diretamente.

Cache com progresso:

```
// index.html — cachedArrayBuffer (linha 1078)
async function cachedArrayBuffer(url, onProgress) {
  // Tenta cache primeiro
  if ('caches' in window) {
    const c = await caches.open(GEO_CACHE_VER);
    const hit = await c.match(url);
    if (hit) { onProgress?.(-1, -1); return hit.arrayBuffer(); }
  }
  // Download com streaming de progresso
  const res = await fetch(url);
  const reader = res.body.getReader();
  const chunks = []; let loaded = 0;
  while (true) {
    const { done, value } = await reader.read();
    if (done) break;
    chunks.push(value); loaded += value.length;
    onProgress?.(loaded, total);
  }
  // Salva no cache para próxima visita
  _geoStore(url, buf.buffer);
  return buf.buffer;
}
```

5.3 Renderização de Eventos (loadXmlContent)

A função `loadXmlContent()` (linha 2389 de `index.html`) é o ponto central

onde um XML de evento é transformado em visualização 3D:

```
// index.html — loadXmlContent (simplificado, linha 2389)
async function loadXmlContent(xmlText, filename){
  // 1. Processar XML via WASM → JSON de hits
  let result;
  try { result = JSON.parse(process_event(xmlText)); }
  catch(err) { showToast(t('parseError')+': '+err.message, true);
return false; }
  if (!result.ok) { showToast(t('invalidXml'), true); return
false; }

  // 2. Atualizar estatísticas no painel
  eMin_mev = result.e_min;
  eMax_mev = result.e_max;

  // 3. Limpar cena anterior
  instGroups.forEach(g => { scene.remove(g.im); g.im.dispose();
});
  instGroups = []; allHits = [];

  // 4. Agrupar hits por geometria (UUID do BufferGeometry)
  const geoGroups = new Map();
  for (const h of result.hits) {
    const mesh = meshIndex[h.path];     // Busca mesh pelo
caminho CGV
    if (!mesh) { _miss++; continue; }   // Mesh não encontrado:
unmapped
    const uid = mesh.geometry.uuid;
    if (!geoGroups.has(uid)) geoGroups.set(uid, []);
    geoGroups.get(uid).push({mesh, h});
  }

  // 5. Criar InstancedMesh para cada grupo de geometria
  for (const [, group] of geoGroups) {
    const geo = group[0].mesh.geometry;
    const mat = new THREE.MeshPhongMaterial({side:
THREE.DoubleSide, shininess: 80});
    const im  = new THREE.InstancedMesh(geo, mat, group.length);
    im.frustumCulled = false;

    for (let i = 0; i < group.length; i++) {
      const {mesh, h} = group[i];
      // Matriz de transformação do mesh original
      im.setMatrixAt(i, mesh.matrixWorld.clone());
      // Cor baseada na energia (colormap)
      const col = colormapColor(h.energy);
      im.setColorAt(i, new THREE.Color(col.r, col.g, col.b));
```

```
    }
    scene.add(im);
    instGroups.push({im, hits, mat});
  }

  // 6. Renderizar jets (cones 3D)
  const jetCollections = parseJetsFromXml(xmlText);
  buildJetCones(jetCollections, geoScale, geoCenter);

  return true;
}
```

Conceito de InstancedMesh: Em vez de criar um `THREE.Mesh` separado para cada hit (o que seria milhares de draw calls), os hits que compartilham a mesma geometria são agrupados em um único `THREE.InstancedMesh`. Cada instância tem sua própria matriz de transformação e cor, mas compartilha a geometria no GPU. Isso permite renderizar milhares de hits com poucos draw calls.

5.4 Sistema de Colormaps

O sistema suporta 5 mapas de cores, cada um definido como lista de stops (pontos de interpolação):

```
// index.html (linha 1428)
const COLORMAPS = {
  thermal: [
    {t:0, r:.13, g:.40, b:.87},   // Azul escuro
    {t:.25, r:.13, g:.80, b:.87}, // Ciano
    {t:.5, r:.27, g:.87, b:.27},  // Verde
    {t:.75, r:.87, g:.87, b:.13}, // Amarelo
    {t:1, r:.87, g:.17, b:.13}    // Vermelho
  ],
  cool:    [...],
  hot:     [...],
  viridis: [...],
  plasma:  [...],
};

// Interpolação entre stops
function colormapColor(e_mev) {
  const range = cMax - cMin;
  const t2 = Math.max(0, Math.min(1, (e_mev - cMin) / range));
  // Encontra os dois stops adjacentes e interpola linearmente
  const a = stops[i], b = stops[i+1];
  const s = (t2 - a.t) / (b.t - a.t);
  return {
    r: a.r + s * (b.r - a.r),
    g: a.g + s * (b.g - a.g),
    b: a.b + s * (b.b - a.b)
  };
}
```

O colormap é aplicado durante a criação dos InstancedMesh e pode ser atualizado em tempo real via `recolorAll()`. O usuário pode definir valores E min/max customizados ou usar auto (min/max do evento).

5.5 Interação e Seleção de Células

O viewer implementa um sistema completo de interação com as células:

- **Hover:** Raycast a cada 3 frames (`updateHover()`), cria clone com material amarelo semi-transparente
- **Seleção:** Clique esquerdo seleciona célula, mostra overlay vermelho
- **Exclusão:** Tecla Del remove célula selecionada (esconde via matriz zero)
- **Undo/Redo:** Ctrl+Z / Ctrl+Shift+Z com pilha de histórico
- **Restauração:** Botão "Restore all" restaura todas as células excluídas

- **Threshold:** Slider que oculta células abaixo de um limiar de energia

A exclusão funciona definindo a matriz da instância para `makeScale(0,0,0)`, efetivamente colapsando o mesh a um ponto invisível.

5.6 Ghost Atlas e Elementos Auxiliares

A função `buildGhostAtlas()` (linha 1544) cria uma representação wireframe dos sub-detectores internos que não são renderizados como geometria sólida. Isso fornece contexto espacial ao usuário:

```
// index.html — buildGhostAtlas (simplificado, linha 1544)
function buildGhostAtlas(scale, center) {
  // Inner Detector (r=1100mm, z=±3100mm)
  cyl(1100, -3100, 3100, 32, wI);
  // Solenoid (r=1275mm, z=±2650mm)
  cyl(1275, -2650, 2650, 32, wI);
  // LAr EM Barrel (r=1422-1985mm, z=±3200mm)
  cyl(1422, -3200, 3200, 32, wLAr);
  cyl(1985, -3200, 3200, 32, wLAr);
  // TileCal Barrel (r=2300-3820mm, z=±2820mm)
  cyl(2300, -2820, 2820, 64, wTc);
  cyl(3820, -2820, 2820, 64, wTc);
  // ... Extended Barrel, MBTS, Endcap, HEC, FCal
}
```

Cada sub-detector usa um material de cor distinta com baixa opacidade (4-25%) para não obstruir a visualização dos hits.

5.7 Jets (Cones 3D)

Jets são parseados do JiveXML via `DOMParser` e renderizados como cones 3D orientados na direção (η, φ):

```
// index.html — buildJetCones (simplificado, linha 1777)
for (const jet of coll.jets) {
  // Converter (η, ϕ) para direção 3D
  const theta = 2 * Math.atan(Math.exp(-jet.eta));
  const dx = Math.sin(theta) * Math.cos(jet.phi);
  const dy = Math.sin(theta) * Math.sin(jet.phi);
  const dz = Math.cos(theta);

  // Comprimento proporcional ao ET (escala logarítmica)
  const coneLen = Math.max(200, Math.min(3000,
    200 + 400 * Math.log10(1 + etGeV))) * scale;
  const coneRadius = coneLen * Math.tan(jetR) * 0.5;

  // Orientar cone do IP na direção do jet
  const dir = new THREE.Vector3(dx, dy, dz).normalize();
  const quat = new THREE.Quaternion().setFromUnitVectors(
    new THREE.Vector3(0, 1, 0), dir
  );
}
```

6. Sistema de Eventos ao Vivo

6.1 LivePoller

A classe `LivePoller` (`xml_test/live_cern/live_poller.js`) é uma máquina de estados que busca eventos do servidor ATLAS a cada 5 segundos:

```
┌──────────┐        ┌──────────┐        ┌──────────┐
│  polling  │──ok──▶│downloading│──ok──▶│   same    │
│ (fetch /  │        │ (fetch    │        │ (espera   │
│  latest)  │        │  XML)     │        │ próximo) │
└──────┬───┘        └──────┬───┘        └──────┬───┘
       │ fail               │ fail              │
┌──────▼───┐        ┌──────▼───┐              │
│  error    │        │  error    │              │
│ (retry    │        │ (retry    │              │
│  5s)      │        │  5s)      │              │
└──────────┘        └──────────┘              │
                                              │ 5s
                                       ┌──────▼──┐
                                       │  polling  │
                                       │ (novo ciclo)│
                                       └──────────┘
```

Ciclo de polling (_poll):

1. **Fetch /latest:** Baixa a página HTML de `atlas-live.cern.ch/latest` via proxy CORS
2. **Parse HTML:** Extrai a referência JiveXML via regex: `src="([^"]*JiveXML_(\d+)_(\d+)\.png)"`
3. **Comparação:** Se o eventId é novo (diferente do último), baixa o XML correspondente
4. **Armazenamento:** Adiciona à lista interna e salva no Cache API do browser
5. **Eventos:** Emite `newxml` para o viewer carregar o novo evento

6.2 CORS Proxy

Como o servidor ATLAS não habilita CORS, o sistema utiliza uma cadeia de proxies:

PRIORIDADE	PROXY	TIPO	DESCRIÇÃO
1	Cloudflare Worker	qs (?url=)	Próprio, mais confiável
2	Direct	raw	Funciona na rede CERN
3	AllOrigins	json	Proxy público (resposta JSON)
4	CodeTabs	raw	Proxy público alternativo
5	HTMLDriven	raw	Último recurso

O Cloudflare Worker (`xml_test/live_cern/cors-proxy.js`) é restrito a `atlas-live.cern.ch`:

```
// cors-proxy.js (linha 41)
if (targetUrl.hostname !== ALLOWED_ORIGIN_HOST) {
  return jsonError(403, `Forbidden: only ${ALLOWED_ORIGIN_HOST}
is allowed`);
}
```

Quando um proxy funciona, ele é "travado" (`_proxyIdx`) para evitar re-probing em cada ciclo.

6.3 Cache de Eventos

O `LivePoller` utiliza a **Cache API** do browser para persistir eventos entre sessões:

- Cache name: `atlas-live-xml-v2`
- Limite: **10 eventos mais recentes**
- Metadata (id, timestamp, url) armazenado no header `X-Atlas-Meta`
- Na inicialização, eventos são restaurados do cache e ordenados por timestamp
- Ao exceder o limite, os eventos mais antigos são removidos

6.4 Modo Offline

O modo offline permite carregar arquivos XML locais de duas formas:

1. **showDirectoryPicker API:** Seleciona um diretório inteiro e lista todos os

`.xml`

2. **File input fallback:** Seleciona arquivos individuais

O carousel de navegação permite navegar entre eventos com setas ← →, e o auto-play avança automaticamente a cada 8 segundos.

7. Delivery dos Arquivos

7.1 Estratégia de Cache

O sistema implementa **duas camadas de cache** independentes:

1. Cache de Geometria (permanente)

RECURSO	CACHE KEY	ESTRATÉGIA
CaloGeometry.glb	`cgv-geo-v1`	Cache-first com versionamento. Bump `GEO_CACHE_VER` para invalidar.
CaloGeometry.cgv	`cgv-geo-v1`	Mesmo versionamento do GLB

```
// Purga automática de versões antigas
if ('caches' in window) {
  caches.keys().then(ks => ks.forEach(k => {
    if (k.startsWith('cgv-geo-') && k !== GEO_CACHE_VER)
caches.delete(k);
  }));
}
```

2. Cache de Eventos (rotativo)

CACHE	LIMITE	ESTRATÉGIA
`atlas-live-xml-v2`	10 eventos	FIFO — remove o mais antigo quando excede

7.2 Hosting e CDN

O projeto é hospedado via **GitHub Pages** com a seguinte estratégia de delivery:

ARQUIVO	ORIGEM	RAZÃO
index.html, JS, CSS	GitHub Pages	Texto leve, CORS automático
CaloGeometry.cgv	GitHub Pages	Texto leve (~600KB)
CaloGeometry.glb	media.githubusercontent.com (Git LFS)	Binário pesado (~50MB), CORS habilitado
WASM (.wasm, .js)	GitHub Pages (relativo)	Carregado via import local
Three.js, fontes, ícones	CDN (jsdelivr.net, googleapis)	Cache CDN global

7.3 Validação de Integridade

Antes de usar o GLB, o sistema verifica os **magic bytes** do formato GLTF 2.0:

```
// index.html (linha 2823)
const h = new Uint8Array(buf, 0, 12);
const ok = h[0]===0x67 && h[1]===0x6C && h[2]===0x54 &&
h[3]===0x46; // 'glTF'
if (!ok) throw new Error('Not a valid GLB binary');
```

Para o CGV, a validação é feita pelo WASM que conta linhas terminando em `cell_*`:

```
// wasm/src/lib.rs — load_cgv (linha 434)
pub fn load_cgv(cgv_text: &str) -> usize {
    cgv_text.lines()
        .filter(|l| {
            let t = l.trim();
            !t.is_empty() && !t.starts_with('#') && {
                let last = t.split('\t')
                    .filter(|s| !s.trim().is_empty() && *s !=
"→")
                    .last();
                last.map(|s|
s.starts_with("cell")).unwrap_or(false)
            }
        })
        .count()
}
```

8. Restrições, Validações e Tolerâncias

8.1 Restrições na Conversão (root2scene.mjs)

RESTRIÇÃO	IMPLEMENTAÇÃO	EFEITO
Visibilidade	`geoAtt & 0x08` (bit kVisThis)	Nós invisíveis são pulados
Profundidade	`--depth <n>`	Limita profundidade da árvore
Faces por shape	`--max-faces <n>`	Limita tesselação de cada geometria
Subtree	`--subtree <prefix>`	Filtra filhos diretos da raiz por prefixo
Identidade	`TGeoIdentity → null`	Matrizes identidade não geram transformação
Shape desconhecida	Key única com random	Sem compartilhamento de geometria

8.2 Tolerâncias no Matching de Hits (WASM)

SUB-DETECTOR	TOLERÂNCIA H	DETALHES
TileCal	min_dist + 0.0005	Tolerância estrita baseada na distância mínima encontrada
LAr Barrel	half_deta(layer) + 0.002	Variável por camada: 0.0016 (S1) a 0.025 (S3)
LAr Endcap	half_deta(layer, i) + 0.003	Variável por camada E índice (segmentação não-uniforme)
HEC	0.08	Threshold fixo para η
Seleção de candidatos	best_dist + 0.001 a 0.005	Filtragem pós-seleção do melhor candidato

8.3 Validações no Viewer (index.html)

- **GLB:** Verificação de magic bytes antes do parse

- **XML:** Resultado do WASM verificado via `result.ok`
- **Meshes:** Hits sem mesh correspondente são contados como `_miss` e logados
- **Energia:** Interpolação de colormap clampada a [0, 1]
- **Cache:** Versões antigas purgadas automaticamente
- **Proxy:** Timeout de 15 segundos por tentativa, fallback sequencial

8.4 Restrições no XML (WASM)

- **Arrays desiguais:** Truncados ao menor comprimento
- **Tokens inválidos:** `parse::<f32>()` com `filter_map` — silenciosamente ignorados
- **Tags ausentes:** Bloco inteiro é pulado (`Option::None`)
- **Hit sem match:** Incrementa `unmapped`, não gera erro
- **Channel MBTS inválido:** `_ => continue` — pula o hit
- **η fora da cobertura:** LAr Barrel rejeita |η| > 1.55, Endcap rejeita fora de [1.35, 3.25], HEC rejeita fora de [1.4, 3.3]

9. Setup e Utilitários

9.1 Instalação e Configuração

O script `setup.mjs` prepara as dependências JSROOT patcheando os imports internos:

```
// setup.mjs
// Copia geobase.mjs e csg.mjs de node_modules/jsroot/modules/
geom para lib/
// e substitui imports internos pelos exports públicos do JSROOT:

patch('geobase.mjs', [
  ["from '../core.mjs'",        "from 'jsroot/core'"],
  ["from '../base/colors.mjs'", "from 'jsroot/colors'"],
  ["from '../base/base3d.mjs'", "from 'jsroot/base3d'"],
]);

patch('csg.mjs', [
  ["from '../base/base3d.mjs'", "from 'jsroot/base3d'"],
]);
```

Passos de setup:

```
# 1. Instalar dependências
npm install jsroot --ignore-scripts

# 2. Patchear imports
node setup.mjs

# 3. Converter arquivo .root (exemplo)
node root2scene.mjs CaloGeometry.root --out ./root_file --
visible-only

# 4. (Opcional) Otimizar GLB
node optimize_glb.mjs --quantize

# 5. Compilar WASM (requer wasm-pack)
cd wasm && wasm-pack build --target web
```

9.2 Scripts npm

O `package.json` define o projeto como módulo ESM:

```
{
  "name": "root2scene",
  "version": "1.0.0",
  "type": "module",
  "main": "root2scene.mjs",
  "bin": { "root2scene": "./root2scene.mjs" },
  "dependencies": { "jsroot": "^7.10.3" },
  "devDependencies": {
    "@gltf-transform/core": "^4.3.0",
    "@gltf-transform/functions": "^4.3.0"
  }
}
```

10. Internacionalização

O viewer suporta 4 idiomas com detecção automática do browser:

CÓDIGO	IDIOMA	DETECÇÃO
`en`	English	Padrão
`fr`	Français	`navigator.language` começando com "fr"
`no`	Norsk	"no", "nb" ou "nn"
`pt`	Português	"pt"

As traduções são armazenadas em um objeto `i18n` (linha 1135 de `index.html`) e aplicadas via `data-i18n` attributes nos elementos HTML. A preferência é persistida em `localStorage('cgv-lang')`.

```
// index.html (linha 1348)
function applyLang(code) {
  currentLang = code;
  localStorage.setItem('cgv-lang', code);
  document.querySelectorAll('[data-i18n]').forEach(el => {
    const key = el.getAttribute('data-i18n');
    el.textContent = t(key);
  });
}
```

CGV WEB — Calorimeter Geometry Viewer

UFJF / NIPSCERN · CERN ATLAS TileCal

Documento gerado em Março 2026